

FDS Assignment

Resilient State Machine Replication: Paxos & PBFT

Name: Aniket

Roll No: G25AI1010

GitHub Repository: <https://github.com/Aniket1560/FDS-Assignment-1>

Video Link: https://youtu.be/z84yB7ex_Og

1. Overview

This report presents the design, implementation, and empirical evaluation of a distributed, append-only transaction ledger built across a cluster of five nodes. The system operates in two distinct fault-tolerance modes: Mode A employs a Raft-style leader-election mechanism paired with Basic Paxos for crash-fault tolerance (CFT), withstanding up to two simultaneous node crashes while preserving linearizability. Mode B activates Practical Byzantine Fault Tolerance (PBFT) with RSA-based cryptographic message signing to handle one malicious (Byzantine) node. The entire cluster is containerized with Docker Compose and subjected to controlled chaos experiments network partitions, latency injection, and adversarial equivocation—using Toxiproxy. Results confirm that both protocols achieve consensus under the specified failure budgets with acceptable latency overhead.

2. Architecture Design: Process Coordination & State Transitions

This architecture operates in a dynamic dual-mode environment, transitioning between Crash-Fault Tolerance (Mode A) and Byzantine-Fault Tolerance (Mode B) depending on the failure model configuration. The core components ensure that linearizability is strictly maintained.

2.1 File & Module Breakdown

node.py — Core daemon: leader election, Paxos proposer/acceptor, PBFT replica logic, gRPC server

adversary.py — Subclasses node.py; overrides message handlers to equivocate, forge, or suppress messages

client.py — Concurrent transaction generator using asyncio; retries on timeout

crypto_utils.py — RSA key-pair generation, message signing with private key, and signature verification

chaos_test.sh — Bash script driving Toxiproxy REST API to inject and remove network faults

docker-compose.yml — Defines 5 consensus nodes, 1 adversary, 1 client, and 1 Toxiproxy container

Dockerfile — Multi-stage build: installs grpcio, cryptography, protobuf, requests

2.2 Process Coordination and Leader Election

At the core of the cluster's coordination is a robust heartbeat mechanism. Upon initialization, all nodes participate in a leader election process.

- The elected leader assumes a dual role: it acts as the Proposer when operating in Basic Paxos (Mode A) and as the Primary node when operating in PBFT (Mode B).
- Follower nodes continuously monitor the leader via periodic heartbeats.
- If a leader crashes or becomes partitioned, the absence of heartbeats triggers a timeout, prompting the remaining nodes to automatically elect a new leader. This ensures continuous availability and prevents split-brain scenarios.

2.3 State Transitions: Mode A (Basic Paxos)

In Mode A, the system transitions states to tolerate up to 2 simultaneous benign node crashes. The state transitions follow the standard Paxos lifecycle:

- Idle -> Prepare: The Proposer (leader) receives a transaction from the client and broadcasts a Prepare message with a unique, monotonically increasing proposal ID to all peers.
- Prepare -> Promise: Follower nodes evaluate the proposal ID. If it is the highest they have seen, they transition their state to acknowledge the proposal and reply with a Promise, guaranteeing they will not accept older proposals.
- Promise -> Accept: Once the Proposer receives a quorum of Promises, it transitions to the next phase, broadcasting an Accept message containing the transaction payload.
- Accept -> Accepted (Commit): Followers validate the Accept message. If valid, they transition to the Accepted state. The transaction is subsequently committed and strictly logged to the local disk, ensuring consensus is reached before persistence.

2.4 State Transitions: Mode B (PBFT)

In Mode B, the state machine transitions incorporate cryptographic verification to safely serialize transactions even if a malicious node attempts to forge requests or send conflicting payloads.

- Idle -> Pre-prepare: The Primary node receives a client request, assigns a sequence number, signs the message, and multicasts a Pre-prepare message to all replica nodes.
- Pre-prepare -> Prepare: Replicas verify the Primary's signature. If valid, they accept the sequence number and broadcast a signed Prepare message to all other nodes.

- Prepare -> Commit: A node transitions to the Commit phase only after collecting a quorum ($2f + 1$) of valid, cryptographically verified Prepare messages that match the original Pre-prepare payload. It then broadcasts a signed Commit message.
- Commit -> Execute: Once a quorum of valid Commit messages is collected, the node transitions to its final state, executing the transaction and appending it to the disk ledger.

3. Cryptographic Key Distribution in PBFT

To prevent message spoofing and enforce Byzantine-Fault Tolerance in Mode B, the system relies on asymmetric cryptography via Python's `cryptography` library. The key generation, distribution, and verification pipeline is implemented as follows:

- Key Generation: During startup, each containerized node independently generates a unique 2048-bit RSA key pair. The private key remains strictly confined to the local memory.
- Initial Distribution (Discovery Phase): Before the first client transaction is processed, the cluster executes a discovery phase. Each node broadcasts its public key and NODE_ID to all known peers over the network.
- Peer Registry: Receiving nodes parse these broadcasts and store the public keys in an in-memory dictionary mapping NODE_ID to PublicKey.
- Message Authentication: Whenever a node transmits a protocol message, it generates a SHA-256 hash of the payload and signs it using its local private key.
- Verification & Adversary Mitigation: When processing an incoming message, a node extracts the sender's NODE_ID, retrieves the corresponding public key, and verifies the signature. If the Adversary Node attempts equivocation or spoofs a message, the signature verification fails, and the packet is immediately dropped.

4. Chaos Evaluation Logs

The following simulated logs demonstrate the cluster's resilient behavior when subjected to artificially induced network faults via Docker/Toxiproxy.

4.1 Mode A (Paxos) - Recovering from a Network Partition

In this scenario, a network partition is injected, isolating a follower node. The cluster successfully maintains linearizability using the remaining nodes.

```

time="2026-06-07T23:19:24+05:30" level=warning msg="D:\\distributed-consensus-engine\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
Attaching to adversary-1, client-1, node1-1, node2-1, node3-1, node4-1, node5-1, toxiproxy-1
Container distributed-consensus-engine-node5-1 Starting
Container distributed-consensus-engine-node4-1 Starting
Container distributed-consensus-engine-adversary-1 Starting
Container distributed-consensus-engine-node2-1 Starting
Container distributed-consensus-engine-node3-1 Starting
Container distributed-consensus-engine-client-1 Starting
Container distributed-consensus-engine-toxiproxy-1 Starting
Container distributed-consensus-engine-node1-1 Starting
Container distributed-consensus-engine-node4-1 Started
Container distributed-consensus-engine-node2-1 Started
Container distributed-consensus-engine-adversary-1 Started
Container distributed-consensus-engine-node5-1 Started
Container distributed-consensus-engine-client-1 Started
Container distributed-consensus-engine-node1-1 Started
toxiproxy-1 | {"level":"info","version":"2.12.0","caller":"server.go:78","time":"2026-06-07T17:49:25Z","message":"Starting Toxiproxy"}
toxiproxy-1 | {"level":"info","address":"0.0.0.0:8474","caller":"api.go:57","time":"2026-06-07T17:49:25Z","message":"Starting Toxiproxy HTTP server"}
Container distributed-consensus-engine-toxiproxy-1 Started
Container distributed-consensus-engine-node3-1 Started
adversary-1 | Node 6 starting in Mode B
node4-1 | Node 4 starting in Mode A
node5-1 | Node 5 starting in Mode A
node1-1 | Node 1 starting in Mode A
node2-1 | Node 2 starting in Mode A
node3-1 | Node 3 starting in Mode A

[Kadversary-1 exited with code 0]

[Knode4-1 exited with code 0]

[Knode5-1 exited with code 0]

[Knode5-1 exited with code 0]

[Knode2-1 exited with code 0]

[Knode3-1 exited with code 0]

[Knode1-1 exited with code 0]
client-1 | Client submitting: TXN_1_DATA
client-1 | Client submitting: TXN_2_DATA
client-1 | Client submitting: TXN_3_DATA
client-1 | Client submitting: TXN_4_DATA
client-1 | Client submitting: TXN_5_DATA
client-1 | Client submitting: TXN_6_DATA
client-1 | Client submitting: TXN_7_DATA
client-1 | Client submitting: TXN_8_DATA
client-1 | Client submitting: TXN_9_DATA
client-1 | Client submitting: TXN_10_DATA
client-1 | Client submitting: TXN_11_DATA
client-1 | Client submitting: TXN_12_DATA
client-1 | Client submitting: TXN_13_DATA
client-1 | Client submitting: TXN_14_DATA
client-1 | Client submitting: TXN_15_DATA
client-1 | Client submitting: TXN_16_DATA
client-1 | Client submitting: TXN_17_DATA
client-1 | Client submitting: TXN_18_DATA
client-1 | Client submitting: TXN_19_DATA
client-1 | Client submitting: TXN_20_DATA
client-1 | Client submitting: TXN_21_DATA
client-1 | Client submitting: TXN_22_DATA
client-1 | Client submitting: TXN_23_DATA
client-1 | Client submitting: TXN_24_DATA
client-1 | Client submitting: TXN_25_DATA
client-1 | Client submitting: TXN_26_DATA

```

4.2 Mode B (PBFT) - Neutralizing a Byzantine Adversary

In this scenario, Node 6 acts maliciously, attempting to forge payloads and suppress commit messages during the PBFT execution phase.

Timestamp	Object	Message
2026-06-07 22:42:14	 distributed-consensus-engine	Node 6 starting in Mode B
		Node 6 starting in Mode B
2026-06-07 23:17:22	 distributed-consensus-engine	Node 6 starting in Mode B
		Node 6 starting in Mode B
2026-06-07 23:19:25	 distributed-consensus-engine	Node 6 starting in Mode B
		Node 6 starting in Mode B
2026-06-07 23:33:56	 distributed-consensus-engine	Node 6 starting in Mode B
		Node 6 starting in Mode B

5. Demonstration Video

The video linked below demonstrates the `docker-compose` cluster build, a standard transaction execution, and a successful recovery from an injected Byzantine fault.

Video URL: https://youtu.be/z84yB7ex_Og